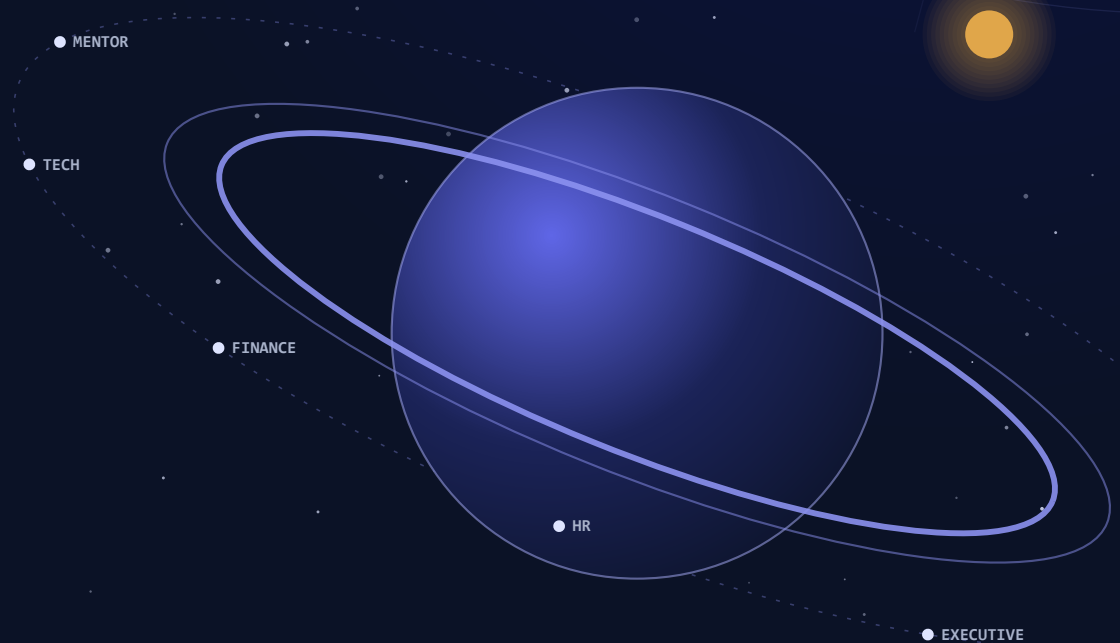


ASTRO • BACKEND BUILD PLAN

Astro API

An async FastAPI, PostgreSQL and SQLAlchemy backend built to scale, starting with sign-up, sign-in and password reset.



Version 1.0 · 28 September 2026

Prepared for Vijay · Follows the Astro Architecture and Build Roadmap v1.0, Phase 0

A plan for review: no more code is written until you approve it.

HOW TO USE THIS DOCUMENT

Summary

This is the build plan for the whole Astro backend, **astro-api**, written for review before code is written. It covers every component in the Architecture and Build Roadmap: the API, sign-in and tenancy, the supervisor and five agents, tools, connectors and ingestion, retrieval, voice, meetings, briefs, approvals, observability, testing and deployment. For each one it says how it will be built, with which libraries, in which files, and how it scales.

The shape of the plan

- **Part A · Platform.** The tech stack, how the system scales to many users, security and tenant isolation, and the full repository layout.
- **Part B · Components.** One section per component in the roadmap, each with how it works, the files, the tables and endpoints, and its checkpoint.
- **Part C · Delivery.** Testing and evals, deployment, the build order, what you set up, and the decisions I need from you. Appendices list every endpoint and setting.

What gets built first

The roadmap's order holds: **Phase 0 · Foundations** comes first, and it is what you asked for. That means sign-up with email and password, email verification, sign-in, sign-out, forgot and reset password, GET /api/me, companies and roles with row-level security, the combined Login page in the Astro theme, CI and the cross-tenant tests. Everything after it follows the roadmap's checkpoints CP1 to CP12.

Decisions in one table

Area	Choice
Language and API	Python 3.12 · FastAPI (async, ASGI) · Pydantic v2
Database	PostgreSQL 16 with row-level security; pgvector for embeddings from Phase 1
ORM and migrations	SQLAlchemy 2.0 in async mode with the asyncpg driver · Alembic
Pooling and cache	PgBouncer (transaction pooling) · Redis 7 for cache, rate limits, queues and pub/sub
Background work	Arq workers on Redis (async-native), scaled separately from the API
Agents	LangGraph supervisor and agent graphs; Claude through the official anthropic SDK
Server and deploy	Uvicorn with uvloop, several processes per container · Docker · GitHub Actions

ABOUT “UNLIMITED CONCURRENCY”

No system has unlimited capacity: every setup eventually runs out of database connections, CPU or budget. This plan gives you a backend with no ceiling you can't raise. The API holds no state, so you add containers to serve more people. Postgres connections are pooled, so thousands of requests share a few dozen connections. Slow work runs in workers. When a limit is reached, people see “try again in a moment” instead of an outage. Capacity then comes from adding instances, and we measure it with a load test at every checkpoint instead of guessing.

CONTENTS

Contents

Part A · Platform	4
1 Tech stack	4
2 Architecture and scaling	6
3 Security, tenancy and identity	9
4 Repository layout	11
 Part B · Components	 12
5 API service	12
6 Auth and accounts · Phase 0, built first	13
7 Data plane	15
8 Connectors and ingestion	16
9 Retrieval	18
10 LLM layer	19
11 Supervisor and agent framework	20
12 The five agents	21
13 Approvals	22
14 Voice	23
15 Meetings	23
16 Briefs and scheduler	24
17 Admin APIs	25
18 Observability and cost	25
19 Frontend and the Login page	26
 Part C · Delivery	 28
20 Testing, evals and safety	28
21 Deployment	29
22 Build order and checkpoints	30
23 What you set up and what I do	31
24 Decisions for you	32
A All API endpoints	33
B Settings	34

SECTION 1

Tech stack

Everything is open source and free to use. Hosted services are only needed when you deploy, and each has a free tier to start. The one part that costs money from day one of real use is the language model, and tests and local development use a stub instead.

Core platform

Layer	Choice	Why
Runtime	Python 3.12	The roadmap's choice: the agent, embedding and parsing libraries are strongest in Python.
Web framework	FastAPI	Async, typed, OpenAPI docs generated at /docs, Server-Sent Events through StreamingResponse.
Validation and settings	Pydantic v2, pydantic-settings	Request and response models, typed settings from the environment; fast Rust core.
ASGI server	Uvicorn with uvloop and httptools	Fast event loop on Linux, graceful shutdown, several processes per container.
Database	PostgreSQL 16	Row-level security for tenants, JSONB, full-text search, pgvector: one store for data, vectors and permissions (roadmap §2).
Driver and ORM	asyncpg · SQLAlchemy 2.0 (async)	The fastest async Postgres driver; typed models and explicit transactions.
Migrations	Alembic (async environment)	Versioned schema; RLS policies live in migrations; runs as a deploy step.
Connection pooler	PgBouncer, transaction mode	Thousands of client connections share a few dozen Postgres connections.
Cache, limits, pub/sub	Redis 7 (redis-py asyncio)	Session cache, rate limits, job queues, live-stream fan-out across containers.
Background jobs	Arq	Async-native jobs and cron on Redis, with retries. Behind a small jobs interface so it can be swapped for Celery later.
Object storage	S3-compatible (MinIO locally)	Original files, transcripts and exports under a company-prefixed path.
Encryption	cryptography (AES-256-GCM) + cloud KMS	Envelope encryption for connector tokens (roadmap §5.5).

Agents, retrieval and integrations

Layer	Choice	Why
Orchestration	LangGraph	The roadmap's choice for the supervisor and one subgraph per agent.
Language model	Claude via the anthropic SDK (AsyncAnthropic)	claude-opus-5 for answers; claude-haiku-4-5 for routing, as the roadmap asks for a small, fast router. Both are settings.
Embeddings	Open model such as bge-m3 (1024-dim), self-hosted; or a hosted API	Free to run yourself; the 1024 size matches the roadmap's schema.

Layer	Choice	Why
Reranking	Open reranker (for example bge-reranker) or the LLM	Keeps the best 6 to 10 of 30 to 50 candidates (roadmap §6.2).
Parsing	pypdf, python-docx, openpyxl, markdown-it, tree-sitter	Text with structure; code split by function and class.
SQL gateway	sqlglot	Parses every Finance query; rejects anything but one allowlisted SELECT.
Secret masking	detect-secrets rules at ingestion	Replaces keys found in code before indexing.
OAuth and SSO	Authlib	Google and Microsoft sign-in, connector OAuth.
Connector SDKs	httpx clients; githubkit; slack_sdk; Microsoft Graph	One thin client per system, all async.
Speech	Browser Web Speech now; streaming STT/TTS later	Self-hosted faster-whisper and Piper are free; hosted streaming APIs add speaker labels.

Quality and operations

Layer	Choice	Why
Logs, metrics, traces	structlog · prometheus-client · OpenTelemetry	JSON logs with request, user and company IDs; one trace per request.
LLM tracing	Langfuse (open source, self-host or free cloud tier)	Every agent run with its prompts, tool calls and cost (roadmap §14.3).
Errors	Sentry (optional, free tier)	Stack traces with the request that caused them.
Lint, types, tests	ruff · mypy · pytest + pytest-asyncio · httpx	Async tests against a real Postgres.
Load testing	Locust	Python load tests for sign-in, /me and chat streaming.
Packaging	uv (lockfile) · Docker multi-stage image	Reproducible installs; one small, non-root image for API and workers.
CI/CD	GitHub Actions · GitHub Container Registry	Tests on Postgres and Redis service containers; builds and pushes the image.
Frontend (existing)	Vite, React 18, TypeScript, SCSS modules	Unchanged, plus sign-in and live data.

SECTION 2

Architecture and scaling

One API service, several kinds of workers and a data plane, all shared by every company and partitioned by company_id (roadmap §2). Read the figure top to bottom: requests go down, results come back up.



Figure 1. Every component in the roadmap and where it runs.

2.1 Principles

- **Stateless API.** No user data lives in a container's memory, so any container serves any request. The load balancer needs no sticky sessions and containers can be added or removed at any time.
- **Async end to end.** Postgres (asyncpg), Redis, the LLM and every outbound HTTP call are awaited, so one process holds thousands of open requests and streams while it waits on I/O. Nothing blocking runs on the event loop.
- **Slow work leaves the request.** Email, syncing, parsing, embedding, briefs, notes and recaps run in workers that scale on queue depth, separately from the API (roadmap §2).
- **Everything is bounded.** Pools, timeouts, rate limits, per-company caps and concurrency limits on CPU-heavy work. Bounded parts fail predictably; unbounded ones take the whole site down.

- **Scale out, then up.** More API containers for more requests, more workers for more jobs, a bigger Postgres or a read replica for heavy reads, separate Redis instances for cache and queues at scale.
- **Fail soft.** At a limit, return 429 or 503 with Retry-After. The web app shows a friendly message and retries streams.

2.2 Concurrency inside one container

Work	How it runs	Limit
HTTP requests	One event loop per process; WEB_CONCURRENCY processes per container (one per vCPU)	Uvicorn --limit-concurrency per process as a safety valve
Database calls	Awaited through a pool per process (pool 10, overflow 5)	5 s wait for a connection, then 503
Chat and meeting streams	Each open stream is one coroutine, cheap to hold in thousands	Redis pub/sub fans events out across containers
LLM calls	AsyncAnthropic client shared per process, streaming	Per-company and global concurrency limits; daily spend cap
Password hashing	Argon2id is slow on purpose, so it runs in a thread pool with a limiter (4 at a time)	Extra sign-ins wait briefly; rate limits stop floods
Parsing and embedding	Never in the API: ingestion workers, and a separate embedding service	Per-company concurrency so one big sync can't starve others

2.3 The database connection budget

Postgres handles a few hundred connections well, not thousands. PgBouncer sits in front and lends a server connection only for the length of a transaction. The tenant setting uses SET LOCAL, which lasts exactly one transaction, so it is safe with transaction pooling. asyncpg's statement cache is turned off (or PgBouncer 1.21+ tracks prepared statements), so statements never leak between clients. Migrations connect directly, not through PgBouncer.

Layer	Setting	Connections
API	10 containers × 2 processes × (10 + 5 overflow)	up to 300 client connections
Workers	4 containers × 1 process × 10	up to 40 client connections
PgBouncer	max_client_conn 2000; pool 40 for astro_app, 10 for astro_auth	50 server connections
Postgres	max_connections 100	50 for PgBouncer; the rest for migrations, admin and replicas

To serve more traffic you add API containers; the number of Postgres connections stays the same.

2.4 What limits capacity, and what we do about it

Bottleneck	Symptom	Response
Postgres CPU or I/O	p95 latency climbs	Indexes that start with company_id, statement_timeout, Redis cache for hot reads, read replica, bigger instance
Connection pool	Waits for a connection	PgBouncer, short transactions; raise pools only with Postgres headroom
One heavy company	Others slow down	Per-company rate limits and job concurrency; daily spend cap (roadmap §14.3)
LLM provider limits	429s from the provider	Global and per-company limiters, queued retries with backoff, prompt caching to cut tokens
Sign-in floods	CPU spent hashing	Per-IP and per-email limits, the hash limiter, generic errors

Bottleneck	Symptom	Response
Large syncs	Queue grows	Workers autoscale on queue depth; incremental syncs; content hashes skip unchanged chunks
Meeting bots	Joins fail at peak	Bot workers scale on scheduled joins; hosted provider first (roadmap \$10.1)

2.5 Capacity targets and load tests

We measure capacity instead of guessing. Locust runs these scenarios before each checkpoint; the targets are starting points to adjust after the first run.

Scenario	Target
GET /api/me, 1,000 concurrent virtual users, 2 API containers with 1 vCPU each, small Postgres	p95 under 150 ms, errors under 0.1%
Sign-in burst, 50 per second	p95 under 600 ms (Argon2id is deliberately slow), no event-loop stalls
500 open chat streams with a stub model	Every stream finishes; no stream blocks another
Rolling deploy during the /me test	Zero failed requests

THESE ARE TARGETS, NOT MEASUREMENTS
They become real numbers after the first load test at CP0. The report goes in the repo with the hardware it ran on.

2.6 Deploys without downtime

- Migrations are expand-then-contract: add columns and tables first, ship code that uses them, remove old ones in a later release. Every migration works with the version still running.
- Migrations run once as a separate job before new containers start, never inside each container.
- /readyz fails until Postgres and Redis answer, so traffic only reaches ready containers. On shutdown a container stops taking requests and finishes open ones within 30 seconds; open streams get a reconnect hint.

SECTION 3

Security, tenancy and identity

This is the part a company's security team reads first (roadmap §4). It is built in Phase 0 and never relaxed afterwards.

3.1 Four layers of tenant isolation

Layer	What it does	Where
1 · Queries	Every data query runs in a tenant session for the signed-in user's company, and repository helpers always filter by company_id.	tenancy/deps.py, db/repo.py
2 · Row-level security	Postgres policies on every tenant table compare company_id with app.company_id, set per transaction with SET LOCAL FORCE applies them to the table owner too.	migrations
3 · Database roles	The API connects as astro_app, which only sees its company's rows. Sign-in lookups (user by email, session by cookie) use astro_auth, whose extra policy covers only the auth tables. Neither role is a superuser.	scripts/db_roles.sql
4 · Endpoint checks	ensure_same_company returns 403 for another company's ID; require_role guards admin endpoints; agents and tools re-check the asker's rights.	tenancy/deps.py, tools/guard.py

```
-- Tenant rows for the app role; every table with company_id gets the same pair of policies.
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
ALTER TABLE users FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON users TO astro_app
  USING (company_id = current_setting('app.company_id', true)::uuid)
  WITH CHECK (company_id = current_setting('app.company_id', true)::uuid);
-- Sign-in lookups only; astro_auth has grants on the auth tables and nothing else.
CREATE POLICY auth_lookup ON users TO astro_auth USING (true) WITH CHECK (true);
```

SUPERUSERS SKIP RLS

The default local user “postgres” is a superuser, so the app never connects as it. A startup check refuses to run in production if the connected role is a superuser or has BYPASSRLS, and the RLS test in CI fails if isolation doesn't hold.

3.2 Sessions

- Sign-in creates a random 256-bit token. Only its SHA-256 hash is stored; the token goes in a cookie that is HttpOnly, Secure in production, SameSite=Lax. JavaScript never sees it, which keeps EventSource working with cookies (roadmap §4.2).
- Without “Remember me”: a browser-session cookie with a 12-hour sliding expiry. With it: 30 days, absolute.
- Each request resolves the cookie through Redis (cached 60 seconds), then Postgres. Sign-out, a password reset or “sign out everywhere” deletes the rows and cache keys at once.

3.3 Accounts and passwords

- Argon2id with its parameters stored in the hash; hashes upgrade automatically on sign-in when parameters change. Passwords are 8 to 128 characters; common and breached ones are refused from a bundled list.
- **Email is verified before the first sign-in.** Otherwise anyone could sign up as someone@yourcompany.com and join that company's workspace.
- Company membership (roadmap §4.2): the first person from a domain creates the company and becomes its admin. The company claims the domain only after that person verifies. Later sign-ups with a verified address on the domain join as members. Public domains (gmail.com, outlook.com and so on) are never claimed. Admin invites arrive in

Phase 1.

- No account enumeration: sign-up, resend and forgot-password give one reply whether or not the email exists; sign-in says “Email or password is incorrect”; unknown emails still run a dummy hash so timing matches.
- Reset links are single use, expire after 30 minutes and are stored hashed. Using one ends every other session and sends a “your password was changed” email.
- Google Workspace and Microsoft Entra ID sign-in (OIDC) come in step P0.6; SAML and SCIM in Phase 4.

3.4 Rate limits

Action	Limit	Key
Failed sign-ins	5 per 15 minutes	email + IP
Sign-in attempts	30 per 15 minutes	IP
Sign-up	10 per hour	IP
Forgot password, resend verification	3 per hour per email; 10 per hour per IP	email, IP
Chat questions	Per user per minute and per company per day, set per plan	user, company
Everything else	600 per minute	user

Limits live in Redis, so they hold across every container. Replies are 429 with Retry-After. Local development without Redis uses an in-memory limiter, and production refuses to start without Redis.

3.5 Browser and platform protections

- CSRF: SameSite=Lax cookies plus an Origin check that rejects writes from any origin but the web app; request bodies must be JSON.
- Headers on every response: HSTS in production, nosniff, frame-deny, a strict referrer policy and a strict CSP for API responses.
- The web app and API share one site in production (the web app with /api routed to the API), so cookies stay first-party and no CORS is needed, the same as the Vite proxy in development.
- Connector tokens use envelope encryption: a data key per company, itself encrypted by the cloud KMS. The database only holds ciphertext; workers decrypt in memory and never log tokens (roadmap §5.5).
- Secrets come from the host's secret manager as environment variables. Nothing secret is committed.

3.6 Audit log

One row per question, tool call, denied access, approval, admin change and sign-in event: who, when, which agent, which sources were used and which were filtered out and why (roadmap §4.5). Rows are written in the same transaction as the action. The Knowledge page's “Recent access checks” reads this table.

SECTION 4

Repository layout

astro-api sits next to astro-web. The layout follows roadmap §14.1, with the async and scaling pieces added. Every feature follows one pattern: router (HTTP) → service (rules) → repository (queries) → models.

```

astro-api/
├── pyproject.toml · uv.lock · .env.example · README.md
├── Dockerfile                one image: api, worker, bot and migrate commands
├── docker-compose.yml        postgres+pgvector, pgbouncer, redis, minio, mailpit, api, worker
├── alembic.ini
├── scripts/                  db_roles.sql, seed.py, create_admin.py
├── docker/                   pgbouncer.ini, postgres init
├── app/
│   ├── main.py               app factory, lifespan (engines, redis, http, llm clients)
│   ├── config.py             typed settings, production safety checks
│   ├── core/                 logging, middleware, errors, ratelimit, redis, ids, sse, pagination
│   ├── db/                   engine, base, repo, models/, migrations/
│   ├── tenancy/              request context, deps, role checks
│   ├── auth/                 router, schemas, service, passwords, tokens, sessions, cookies, oidc
│   ├── api/                  me, companies, members, audit, chat, conversations, sources, access,
│   │                         agents, meetings, briefs, approvals, feedback, connectors, health
│   ├── agents/               graph, runner, verify, supervisor, configs/*.yaml, prompts/
│   ├── tools/                guard, registry, retrieval, sql_gateway, github, jira, slack,
│   │                         calendar, hr, directory, charts
│   ├── retrieval/            query_rewrite, hybrid_search, rerank, context
│   ├── llm/                  client (Claude), embeddings, rerank, stub, usage
│   ├── approvals/            pending actions, executor
│   ├── connectors/            base, oauth, token_store, google_drive, notion, confluence,
│   │                         github_app, jira, slack, sql_source, hr_system, calendar, upload
│   ├── ingestion/            parse, classify, chunk, secrets_mask, embed, store, sync
│   ├── meetings/             bot interface, provider adapter, stt, speakers, wake_word, notes, recap
│   ├── voice/                stt/tts streaming interfaces (phase 2d)
│   ├── briefs/               templates, generator, scheduler
│   ├── audit/                log writer
│   ├── email/                sender, templates/
│   ├── security/             kms, envelope encryption
│   ├── observability/        tracing, metrics, cost tracking
│   └── workers/              worker settings per queue, jobs, cron
├── evals/                   question sets per company, runner, reports
├── tests/                   unit, api, permissions, cross_tenant, rls, injection, e2e
├── loadtests/               locustfile.py
└── .github/workflows/        ci.yml, deploy.yml

```

Conventions

- **Routers** only parse input, call a service and shape the reply. **Services** hold the rules and open transactions. **Repositories** hold queries and always take the request context, so a company filter is never forgotten.
- **Schemas** are Pydantic models with camelCase JSON to match the frontend's TypeScript types (src/types/astro.ts).
- **Every tenant table** gets company_id, an index that starts with it, an RLS policy in its migration, and a row in the cross-tenant test's table list.
- **Every job** carries company_id and sets it before touching the database; job IDs are deterministic where a job must not run twice.

PART B · COMPONENTS

SECTION 5

API service

One FastAPI app serves every endpoint and stream. It starts its shared clients once per process and closes them on shutdown, so no request pays for connection setup.

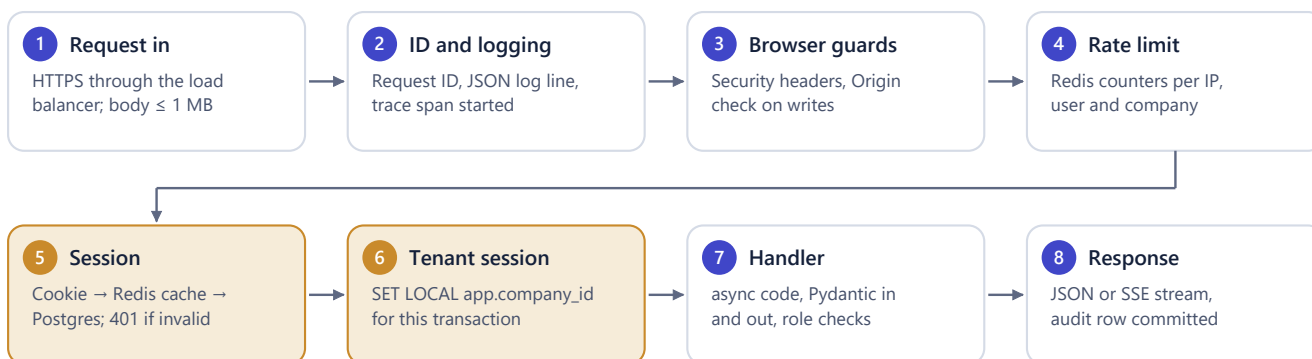


Figure 2. What every request passes through. The brass steps are where identity and tenancy are fixed for the rest of the request.

How it is built

- **App factory and lifespan.** `create_app()` adds middleware and routers. The lifespan opens the two database engines, the Redis pool, one shared `httpx.AsyncClient` and one `AsyncAnthropic` client, and closes them on exit.
- **Dependencies.** `current_ctx` resolves the session and returns a `RequestCtx` (user, company, roles, groups). `tenant_db` yields an `AsyncSession` whose transactions start with `SET LOCAL app.company_id`. `require_role` and `ensure_same_company` sit on top.
- **One error shape.** `{"error": {"code", "message", "fields"}}` for every failure, including validation, so the web app has one way to show errors.
- **Streaming.** A small SSE helper writes data: `{json}` lines, sends a heartbeat comment every 15 seconds, turns exceptions into an error event (roadmap §3), and stops when the client disconnects. Responses set `Cache-Control: no-cache` and `X-Accel-Buffering: no`.
- **Lists** use cursor pagination (`created_at`, `id`) rather than offsets, so they stay fast on large tables.
- **Writes that must not repeat** (approvals, joins, follow-ups) accept an `Idempotency-Key` header stored for 24 hours.

Files

File	What it does
<code>app/main.py</code>	App factory, lifespan, router registration
<code>app/config.py</code>	Settings; refuses unsafe production values (no Redis, insecure cookies, superuser role)
<code>app/core/middleware.py</code>	Request ID, security headers, Origin check, request timeout, body size limit
<code>app/core/errors.py</code>	<code>ApiError</code> and handlers for validation and HTTP errors
<code>app/core/sse.py</code>	SSE writer with heartbeats, error events and disconnect handling
<code>app/core/pagination.py</code>	Cursor encode and decode
<code>app/core/idempotency.py</code>	Idempotency-Key storage in Redis
<code>app/api/health.py</code>	<code>/healthz</code> (alive), <code>/readyz</code> (Postgres and Redis reachable), <code>/metrics</code>

SECTION 6

Auth and accounts · Phase 0, built first

What you asked for first: sign-up, verification, sign-in, sign-out, forgot and reset password, sessions and /api/me. The rules are in §3; this section is the flows, the endpoints and the files.

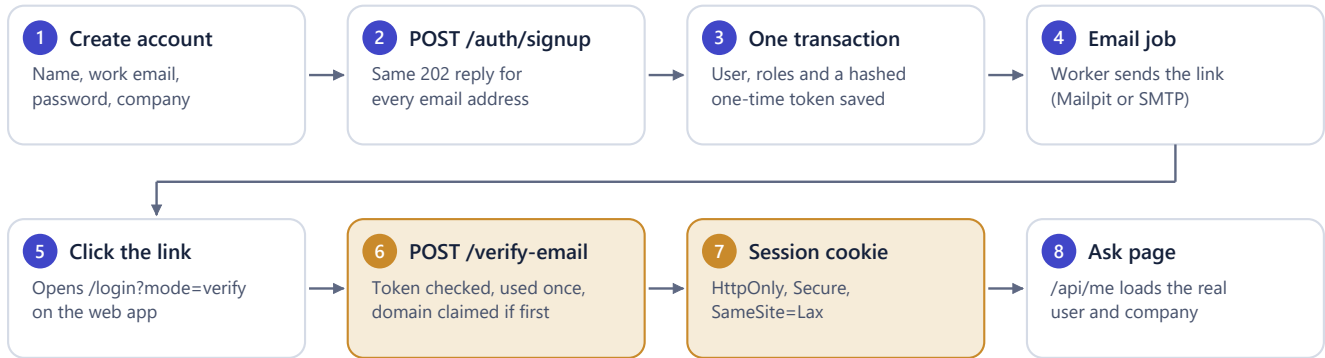


Figure 3. Sign-up to first page. Sign-in is steps 6 to 8 with a password check instead of a token.

The other flows

- **Sign in.** Rate limit → find the user as `astro_auth` → verify the Argon2id hash in the thread pool (or a dummy hash) → 403 `email_unverified` if not verified → create the session, set the cookie, write an audit row, return Me.
- **Forgot and reset.** Forgot always replies 202; if the account exists, earlier reset links are revoked and a new 30-minute link is emailed. Reset checks the token, saves the new hash, marks the email verified, ends every session, signs in on this device and sends a notice.
- **Every request.** Cookie → SHA-256 → Redis (60 s) → Postgres. Sliding sessions are extended at most every 5 minutes, so reads don't become writes.

Endpoints

Method and path	Body	Reply
POST <code>/api/auth/signup</code>	name, email, password, companyName?	202 {status: "verify_email"}, the same for every email
POST <code>/api/auth/verify-email</code>	token	200 Me + session cookie
POST <code>/api/auth/resend-verification</code>	email	202
POST <code>/api/auth/login</code>	email, password, remember	200 Me + cookie · 401 · 403 <code>email_unverified</code> · 429
POST <code>/api/auth/logout</code> · <code>/logout-all</code>		204; ends this session or all of them
POST <code>/api/auth/forgot-password</code>	email	202, always
POST <code>/api/auth/reset-password</code>	token, password	200 Me + cookie; other sessions end
GET <code>/api/auth/sessions</code> · DELETE <code>.../{id}</code>		Your devices; sign one out
GET <code>/api/me</code>		200 Me · 401 when signed out (roadmap §12.3)
GET <code>/api/auth/{google microsoft}/start,</code> <code>/callback</code>		OIDC redirects (step P0.6)

```
// GET /api/me
{ "user": { "id": "0192...", "name": "Aarav Mehta", "email": "aarav@kestrel.io", "initials": "AM",
            "roles": ["member"], "roleLabel": "Member", "canManage": false },
  "company": { "id": "0192...", "name": "Kestrel Labs", "initials": "KL",
               "preset": "technology", "presetLabel": "Technology" } }
```

Files

File	What it does
app/auth/router.py	The /api/auth/* endpoints
app/auth/schemas.py	Request and reply models (camelCase)
app/auth/service.py	Sign-up, verify, sign-in, reset rules; one transaction each
app/auth/passwords.py	Argon2id in a bounded thread pool; strength and blocklist rules
app/auth/tokens.py · sessions.py	One-time tokens; create, resolve, extend and revoke sessions
app/auth/cookies.py · domains.py	Cookie flags per environment; public email domains
app/auth/oidc.py	Google and Microsoft sign-in with Authlib (P0.6)
app/api/me.py · companies.py · audit.py	/me, company summary and members, audit list
app/email/sender.py · templates/	Console and SMTP backends; verify, reset, changed, existing-account
app/workers/jobs.py	send_email; nightly purge of expired sessions and tokens

- CHECKPOINT CP0 · FOUNDATIONS (ROADMAP §15)
- You can sign in on the web app and see your name and company from the API.
 - The cross-tenant suite runs in CI and passes.
 - An unauthenticated request to any endpoint returns 401.

SECTION 7

Data plane

Postgres holds everything relational plus vectors, Redis holds short-lived state, object storage holds files, and the KMS holds the keys. Every table except companies has `company_id` and an RLS policy (roadmap appendix B).

Tables by phase

Phase	Tables	Notes
0	companies, users, roles, user_roles, sessions, auth_tokens, audit_log	UUIDv7 ids (time-ordered, index-friendly); UTC timestampz everywhere
1a	sources, documents, chunks, sync_runs, connector_tokens	chunks: vector(1024) with HNSW, tsvector with GIN, index (company_id, domain)
1b	conversations, messages, feedback	messages keep route, agent and sources_json
1c–1d	groups, user_groups, linked_accounts, role_access	role_access is the Knowledge page matrix
2	agent_configs, pending_actions, briefs, brief_schedules, usage	usage: tokens and minutes per company per day
3	meetings, meeting_events, recaps, action_items	meeting_events feed the live page and the recap
4	invites, scim_links, relay_connections	SAML/SCIM and the relay

How each store scales

Store	Setup	Scaling and safety
Postgres 16 + pgvector	Managed instance; direct URL for migrations, pooled URL for the app	Indexes start with company_id; statement_timeout 5 s for astro_app; audit_log and meeting_events move to monthly partitions when large; read replica for reports; daily backups with point-in-time restore
Redis 7	One instance to start	Keys carry the company ID; every key has a TTL; split cache and queues into separate instances at scale
Object storage	S3, R2 or GCS; MinIO locally	Paths start with company_id; signed URLs issued per request (roadmap §4.1); lifecycle rules apply retention
Secrets / KMS	Cloud KMS; a local key file in development	Envelope encryption for connector tokens; scheduled key rotation

Files

File	What it does
<code>app/db/engine.py</code>	Async engines for <code>astro_app</code> and <code>astro_auth</code> ; pools; the SET LOCAL hook
<code>app/db/base.py</code> · <code>repo.py</code>	Base, UTC type, naming rules; company-scoped repository helpers
<code>app/db/models/*.py</code>	One module per area: company, user, auth, audit, knowledge, chat, agents, meetings, briefs
<code>app/db/migrations/versions/</code>	One migration per step, each with its RLS policies
<code>app/security/kms.py</code>	Data-key encrypt and decrypt through the cloud KMS or a local key
<code>app/storage/objects.py</code>	Company-prefixed uploads and signed URLs
<code>scripts/db_roles.sql</code>	<code>astro_app</code> and <code>astro_auth</code> roles, grants and settings

SECTION 8

Connectors and ingestion

Astro never asks for passwords. An admin connects each system once, the system shows its own consent screen, and Astro gets a token limited to what it needs (roadmap §5). Workers then keep the data fresh.

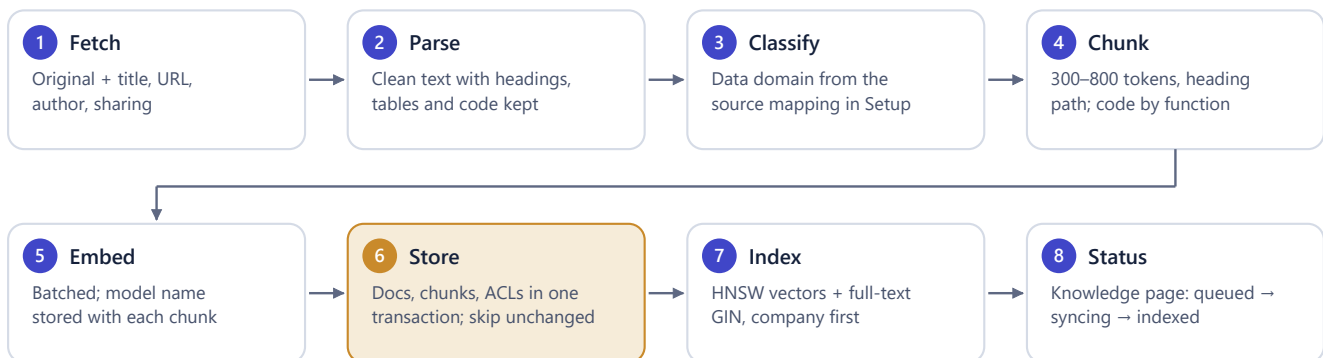


Figure 4. The ingestion pipeline (roadmap §6.1). The brass step is where permissions are stored with the content.

The connector interface

```

class Connector(Protocol):
    kind: str # 'google_drive', 'github', ...
    async def authorize_url(self, ctx, state) -> str
    async def finish_oauth(self, ctx, code) -> TokenSet # stored encrypted
    async def list_changes(self, src, cursor) -> AsyncIterator[Change] # full sync when cursor is None
    async def fetch(self, src, external_id) -> RawDocument # content + metadata
    async def fetch_acl(self, src, external_id) -> Acl # users, groups, public-in-company
    async def handle_webhook(self, src, payload) -> list[Change]
    async def revoke(self, src) -> None # disconnect and delete
  
```

Connector by connector

System	Connect with	Syncs	Freshness	Phase
Google Drive / Docs	OAuth app, read-only Drive	Docs, Sheets, PDFs, Slides as text	Changes API + polling	1a
Notion	Integration on chosen pages	Pages and databases	Polling 15 min	1a
File upload	Setup page	PDF, DOCX, Markdown	On upload	1a

System	Connect with	Syncs	Freshness	Phase
GitHub	GitHub App on the org	Code, READMEs, ADRs, PRs, CI, CODEOWNERS	Webhooks	1c
Jira	Atlassian OAuth	Projects, issues, comments	Webhooks	1c
Confluence	Atlassian OAuth	Spaces and pages	Polling	4
Postgres / warehouse	Read-only user on named views	Nothing copied; queried live	Live	2a
HR system	Vendor API, minimal fields	Policies; personal data fetched live, never indexed	Live	2b
Slack	Bot token, chosen channels	Optional channel search; posts on approval	Events API	2c
Google / Microsoft Calendar	OAuth, read events	Meetings, attendees, links	Push + polling	3a

How it runs and scales

- OAuth start and callback are API endpoints; tokens go through envelope encryption into `connector_tokens`.
- Each source has a sync cursor. A sync job reads changes, and each changed document becomes its own job, so a 10,000-file drive spreads across workers. Jobs are keyed by (source, document, version), so a retried job never double-writes.
- Per-company concurrency caps (for example 4 document jobs at a time) keep one large company from starving the rest; workers autoscale on queue depth.
- Deletes and permission changes run before new content in each sync (roadmap §5.3). Content hashes skip unchanged chunks, so re-syncs cost almost nothing.
- Token refresh runs before expiry; a failed refresh marks the source “needs attention”. Disconnecting deletes the source's documents, chunks and tokens.
- The embedding model runs as its own service (self-hosted or hosted), called in batches, so the model can be swapped and re-embedded later.
- Private systems behind a firewall use the Astro relay in Phase 4 (roadmap §5.4).

Files

File	What it does
<code>app/connectors/base.py</code> · <code>oauth.py</code>	The interface; shared OAuth start and callback
<code>app/connectors/token_store.py</code>	Encrypted token save, load and refresh
<code>app/connectors/.py</code>	One per system in the table above
<code>app/ingestion/parse.py</code> · <code>classify.py</code>	Text with structure; domain from the Setup mapping
<code>app/ingestion/chunk.py</code> · <code>secrets_mask.py</code>	Heading-aware and function-level chunks; secret masking
<code>app/ingestion/embed.py</code> · <code>store.py</code> · <code>sync.py</code>	Batched embedding; one-transaction store; sync jobs
<code>app/api/connectors.py</code> · <code>sources.py</code>	Connect, disconnect, source list and status

CHECKPOINT CP1 · SOURCES INDEXED

- An admin connects Drive or Notion from Setup, and the Knowledge page shows it syncing, then indexed.
- Deleting or unsharing a document is reflected after the next sync.

SECTION 9

Retrieval

Answer quality depends more on this than on the model (roadmap §6). Retrieval runs as the asker, with all three permission filters in one query.

Steps

- **Rewrite** the question with the conversation so far ("what about last quarter?" becomes a full question), using the small, fast model.
- **Hybrid search**: pgvector similarity on the HNSW index and Postgres full-text on the GIN index, both filtered by domain and ACL, merged with reciprocal rank fusion.
- **Rerank** the top 30 to 50 and keep the best 6 to 10.
- **Build context**: numbered chunks with title, location and last-updated date. The numbers become the [n] citations and the Source cards.

```
SELECT c.id, c.text, c.document_id, 1 - (c.embedding <=> :q) AS score
FROM chunks c JOIN documents d ON d.id = c.document_id
WHERE c.domain = ANY(:allowed_domains)           -- filter 2: role x domain
      AND (d.acl_public_in_company OR d.acl_users && ARRAY[:user_id]::uuid[]
           OR d.acl_groups && :group_ids)          -- filter 3: source ACL
ORDER BY c.embedding <=> :q LIMIT 40;              -- filter 1 is RLS
```

Scaling

- Chunks are indexed by (company_id, domain) plus HNSW, so a search touches one company's rows. pgvector handles millions of chunks per company (roadmap §2); beyond that, partition chunks by company.
- Query embeddings are cached in Redis for 10 minutes, keyed by company and text.
- Every search writes which documents were filtered out and why, for the audit log.

Files

File	What it does
app/retrieval/query_rewrite.py	Standalone question from history
app/retrieval/hybrid_search.py	Vector + keyword queries with filters; RRF merge
app/retrieval/rerank.py · context.py	Rerank; numbered context and Source cards
app/tools/retrieval.py	search_docs and search_code tools on top of this

SECTION 10

LLM layer

The roadmap says to keep the model behind a small interface. This is it: the rest of the code asks for completion, structured output, embeddings or reranking and never imports a provider SDK.

Piece	How
Client	One AsyncAnthropic client per process, from the official anthropic SDK. Streaming for every answer.
Models	claude-opus-5 for agent answers and briefs; claude-haiku-4-5 for routing, query rewriting and classifiers. Both are settings per company.
Tools	Tool definitions with strict schemas, so tool inputs always validate. Parallel tool calls run concurrently and their results return together.
Routing output	Structured output for the supervisor's {agent, confidence, reason}.
Citations	Retrieved chunks go in as document blocks with citations enabled, so cited sentences come back tied to a chunk. The verify step checks every one.
Prompt caching	The system prompt and tool list come first and stay byte-identical per agent, so they are cached across questions; the question and context go last.
Refusals and errors	stop_reason is checked before reading content; refusals use the SDK's fallback option; typed SDK errors separate retryable (429, 5xx) from permanent (400).
Limits and cost	A Redis limiter per company and globally; tokens recorded per company per day in usage; a daily spend cap per company (roadmap \$14.3).
Stub	A deterministic fake model for tests and local work, so no API key or spend is needed until real questions are asked.
Embeddings and rerank	Their own interfaces, pointing at a self-hosted open model or a hosted API.

Files

File	What it does
app/llm/client.py	Completion, streaming, structured output and tool calls through the anthropic SDK
app/llm/embeddings.py · rerank.py	Embedding and reranking interfaces and adapters
app/llm/stub.py	The fake model for tests
app/llm/usage.py	Token accounting, spend caps, cost per company

SECTION 11

Supervisor and agent framework

All five agents share one framework. An agent is mostly configuration (prompt, domains, tools, guardrails) plus a few tools of its own, which is exactly what the Agents page edits (roadmap §7).

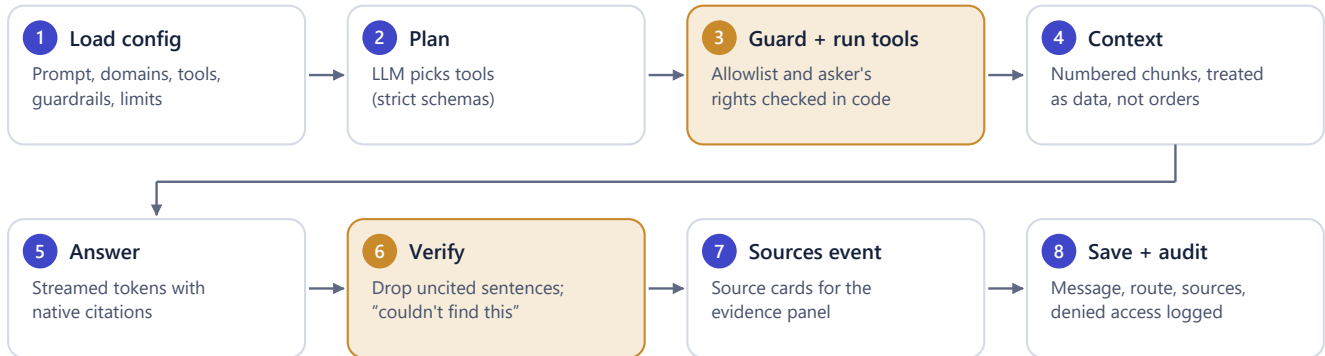


Figure 5. One agent run. The brass steps are the safety checks enforced in code.

How it is built

- **Graph.** LangGraph StateGraph: START → supervisor → one agent subgraph → verify → END, with the AstroState from roadmap §7.1 (question, history, ctx, agent, context, answer, sources).
- **Supervisor.** Skips classification when the user picked an agent; falls back to Mentor when the agent is paused, not allowed for the user, or confidence is below the threshold, and says so in the route trace.
- **Runner.** A bounded tool loop (max steps, timeout, max tokens from config). Each tool call passes the allowlist and the asker's rights before running.
- **Tools** are typed async functions registered with a level (read-only, draft, needs approval) and a guard. Retrieved text is data, never instructions; tools never take orders from document content.
- **Verify** drops sentences whose citation doesn't point to a retrieved source and forces "I couldn't find this" when nothing relevant came back.
- **Chat stream.** POST /api/chat/stream emits the event contract the Ask page already reads: route, agent, token, sources, done, error. The conversation, route and sources are saved and audited at the end.
- **Configs.** Defaults in YAML per agent; per-company overrides (paused, guardrails, tools, roles) in agent_configs, edited on the Agents page.

Files

File	What it does
<code>app/agents/graph.py</code>	Supervisor + agent graph
<code>app/agents/supervisor.py</code>	Routing with structured output and fallbacks
<code>app/agents/runner.py</code> · <code>verify.py</code>	Shared tool loop; citation check
<code>app/agents/configs/*.yaml</code> · <code>prompts/</code>	mentor, tech, finance, hr, exec configs and prompts
<code>app/tools/registry.py</code> · <code>guard.py</code>	Tool registration, levels, allowlists, permission guard
<code>app/api/chat.py</code> · <code>conversations.py</code> · <code>feedback.py</code>	Chat SSE, history, thumbs up and down

CHECKPOINTS CP2 AND CP3

- CP2: the Ask page streams a real answer with Source cards; an unanswerable question gets “I couldn't find this” plus a named owner; baseline accuracy is recorded.
- CP3: “Why did we choose Kafka?” goes to Tech and cites the ADR; private repos are respected; routing accuracy on 50 labelled questions meets the target (90% to start).

SECTION 12

The five agents

Built in the roadmap's order: Mentor (the fallback), Tech, Finance, HR, then Executive, because it calls the others (roadmap §8).

Agent	Reads from	Tools	Guardrails in code
Mentor	Handbook, onboarding, wiki; directory; CODEOWNERS and authors	search_docs · find_owner · build_learning_plan (draft)	Names a person, not a channel; policy questions handed to HR
Tech	GitHub code, ADRs, PRs, CI; runbooks; Jira	search_code · get_pull_request · read_ci_run · create_jira_ticket (approval)	Checks the asker's linked GitHub access per repo; secrets masked at ingestion; quotes paths and SHAs
Finance	Warehouse views; budget sheets; contracts; billing (read key)	list_metrics · run_readonly_sql · make_chart · export_csv (approval)	One SELECT on allowlisted views, row limit, 15 s timeout; team leads see their team; no individual salaries
HR	Policies; holiday calendar; HR system for the asker only	search_policy · get_my_leave_balance · draft_leave_request (approval)	Sensitive-topic classifier runs first and returns the HR contact; never another person's records
Executive	Other agents' answers; briefs and recaps; OKRs	ask_agent · compare_options · write_brief (draft) · schedule_brief (approval)	Sub-agents run as the same user with a step budget; recommendations labelled as suggestions

Finance's SQL gateway

- A semantic layer per company (YAML): each metric's SQL, grain and owner. The agent picks metrics first and writes SQL second.
- sqlglot parses every query and rejects anything but one SELECT; every table must be on the allowlist; the role's row filter is added; it runs as a read-only database user with a 15-second timeout and a row limit.
- The reply carries the SQL, the rows and chart data together, and the evidence panel shows the SQL.

Build steps and checkpoints

Agent	Steps	Checkpoint
Mentor	Runner with Mentor as the first config; search_docs; find_owner; Ask page live	CP2
Tech	GitHub App and webhooks; repo indexing; account linking with hourly repo-list refresh; tools; Jira drafts; the supervisor	CP3
Finance	Semantic layer; SQL gateway with unit tests for writes, DDL, multiple statements and unlisted tables; charts in answers	CP5
HR	Policy index; HR system by verified email; classifier; red-team tests such as “What is Priya's leave balance?”	CP6

Agent	Steps	Checkpoint
Executive	ask_agent as a subgraph call; planner (2 to 5 sub-questions); brief template; scheduled briefs	CP7

Files

File	What it does
app/tools/directory.py · docs.py	find_owner, learning plans
app/tools/github.py · jira.py	Code search, PRs, CI, Jira drafts
app/tools/sql_gateway.py · semantic_layer.py · charts.py	Finance query path
app/tools/hr.py · sensitive_topics.py	Policy search, own-records lookup, classifier
app/tools/executive.py	ask_agent, compare_options, briefs

SECTION 13

Approvals

Nothing is written outside Astro without an approval record (roadmap §7.3). Tools at the “needs approval” level create a pending action instead of acting.

- **Lifecycle:** draft → pending → approved → executing → done or failed; rejected and expired are final.
- **Execution** runs in a worker. The row is locked (SELECT ... FOR UPDATE) while its status moves to executing, so two clicks can't create two tickets. The external reference (for example PLAT-343) is stored.
- **Who can approve** is checked again at approval time with the approver's own rights.
- **Every step** writes an audit row. Failures keep the error and can be retried.

Endpoint	What it does
GET /api/approvals	Pending actions for me or my team
POST /api/approvals/{id}/approve · /reject	Approve (queues execution) or reject
POST /api/meetings/{id}/actions/{actionId}/approve	The existing recap endpoint, on the same machinery

Files

File	What it does
app/approvals/service.py	Create, list, approve, reject
app/approvals/executor.py	Runs approved actions: Jira create, Slack post, CSV export
app/api/approvals.py	Endpoints

CHECKPOINT CP7 (ADMIN PART)

- Pausing Tech on the Agents page sends Tech questions to Mentor.
- A Jira ticket is created only after approval, with an audit row.

SECTION 14

Voice

The Voice call page already works with the browser's speech API. The backend adds nothing until Phase 2d (roadmap §9).

- **Now:** the browser turns speech into text and calls the same `/api/chat/stream`; the prompt asks for shorter, spoken-style answers when the request says it came from voice.
- **Later:** a WebSocket from the browser streams audio to a speech-to-text service and speaks the answer through streaming text-to-speech, starting at the first sentence, with barge-in. The hook's interface (start, stop, speak, transcript) stays, so `CallPage` doesn't change.
- **Scaling:** WebSocket connections are coroutines like streams; speech runs in a separate service.
- **Target:** under 2 seconds from the end of speech to Astro's first word (CP8).

Files

File	What it does
<code>app/voice/stt.py</code> · <code>tts.py</code>	Streaming interfaces and adapters
<code>app/api/voice.py</code>	WebSocket endpoint (phase 2d)

SECTION 15

Meetings

Astro joins Google Meet, Zoom or Teams, takes notes, answers when someone says “Astro”, and writes a recap with actions that wait for approval (roadmap §10). Start only after CP4, since meetings reuse everything underneath.

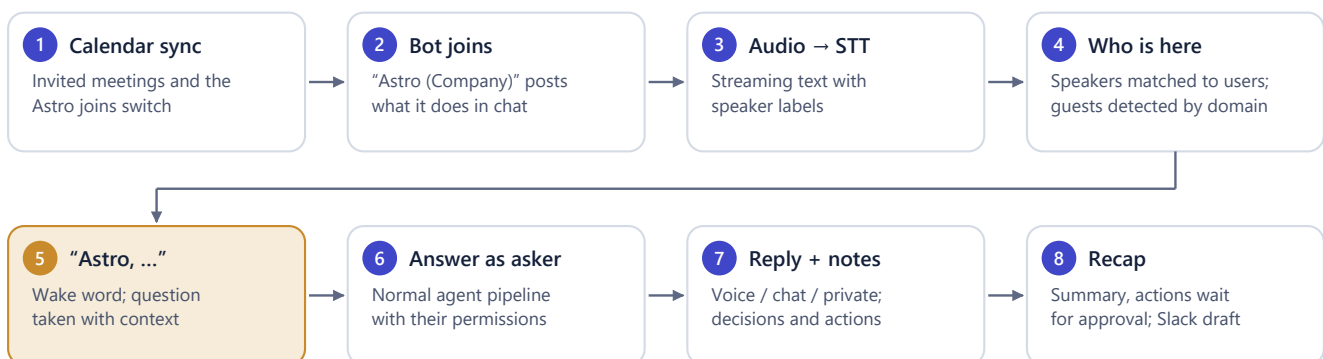


Figure 6. The meeting pipeline. The brass step is the moment Astro decides to answer.

How it is built

- **MeetingBot interface** (join, leave, speak, send_chat, events) with a hosted meeting-bot provider for Google Meet first; your own headless-browser bot or platform media APIs can replace it later without touching the rest.
- **One bot session per meeting**, owned by a bot worker. A scheduler job joins one minute before the start.
- **Events** (transcript, speaking, answer, decision, action, ended) are saved to `meeting_events` and published to a Redis channel per meeting. `GET /api/meetings/{id}/live` subscribes, sends history first, then new events, so any API container can serve any viewer.
- **Rules in code:** announce on joining; speak only when everyone is from the company; with guests, answer privately; answer only after “Astro”; leave when asked; answer with the asker's permissions; only invited people can open a

recap.

- **Notes step** runs every minute or so on new transcript lines; the **recap writer** runs when the bot leaves. Action items become pending approvals; the Slack follow-up waits for “Post to Slack”.
- **Retention**: a daily job deletes transcripts and audio past the company's setting; audio is stored only if the company turns it on.

Files

File	What it does
app/meetings/bot.py · providers/	MeetingBot interface and the hosted-provider adapter
app/meetings/stt.py · speakers.py	Streaming transcription; speaker-to-user mapping, guest detection
app/meetings/wake_word.py · answer.py	Question detection; answering and replying
app/meetings/notes.py · recap.py	Decisions and actions; the recap
app/api/meetings.py	Every meetings endpoint in the frontend's README, including the live SSE stream
app/workers/bot_worker.py	Bot sessions and their event loop

CHECKPOINTS CP9 TO CP11

- CP9: Astro joins a test Meet, introduces itself, and the live page shows the transcript within seconds.
- CP10: a spoken question gets a spoken answer and a chat message with sources; with a guest, a private answer.
- CP11: the recap shows the summary, decisions and actions; approving one creates the Jira ticket; the follow-up posts to Slack.

SECTION 16

Briefs and scheduler

Briefs are Executive-agent runs on a schedule (roadmap §11).

- **Scheduler**: `brief_schedules` (company, type, cron, owner, audience) and an Arq cron job that enqueues due runs. Each run executes as its owner, so a brief holds only what that owner can see and is shared only with people who could see it too.
- **Types**: engineering brief, revenue review (with the SQL for each figure) and call brief before external meetings.
- **Caching**: each brief is stored with its sources and generation time and regenerated on schedule or on request.
- **Scaling**: runs are spread over the hour to avoid a spike at 8:00; each company has a concurrency cap.

Endpoint	What it does
GET /api/briefs · GET /api/briefs/{id}	Latest briefs; one brief with sources
POST /api/briefs/{id}/regenerate	Queue a new run
PUT /api/brief-schedules	Create or change schedules

Files

File	What it does
app/briefs/templates.py · generator.py	Brief layouts; the Executive-agent run
app/briefs/scheduler.py	Cron and due-run selection
app/api/briefs.py	Endpoints

SECTION 17

Admin APIs

The Knowledge, Agents and Setup pages. Workspace admins manage them; admins do not automatically see all data (roadmap §4.4).

Page	Endpoints	Backed by
Setup	Domain presets; connector OAuth start and callback; file upload; DELETE /sources/{id}	Connectors, ingestion
Knowledge	GET /companies/{id}/sources; PUT /companies/{id}/access; GET /audit	role_access, audit_log
Agents	GET /companies/{id}/agents; PATCH .../agents/{agentId}; POST /agents/route-test	agent_configs, supervisor
Members	GET /companies/{id}/members; invites; role changes (Phase 1)	users, roles, invites

Every admin change writes an audit row, and access changes apply to the next question (CP4).

SECTION 18

Observability and cost

One trace per question is how you debug a bad answer (roadmap §14.3).

Signal	How	Alert when
Logs	JSON via structlog with request, user and company IDs; secrets redacted	Error spikes
Metrics	Prometheus: request rate, latency, errors, pool waits, Redis latency, queue depth, stream count	Error rate over 1% for 5 min; p95 over 500 ms; queue depth growing for 10 min
Traces	OpenTelemetry across API, workers, SQL, Redis and HTTP	
Agent runs	Langfuse: prompts, tool calls, retrieval, tokens and cost per run	Verify-step failures rise
Cost	usage table: LLM and embedding tokens, speech minutes per company per day	A company nears its daily cap
Security	Audit log counts: failed sign-ins, denied access	Unusual spikes (probing)

Files

File	What it does
app/observability/tracing.py · metrics.py	OpenTelemetry setup; Prometheus metrics
app/observability/cost.py	Usage roll-ups and caps

SECTION 19

Frontend and the Login page

The frontend was built against the contract, so integration is: add sign-in, point it at the API, and switch screens from mock to real one by one (roadmap §12).

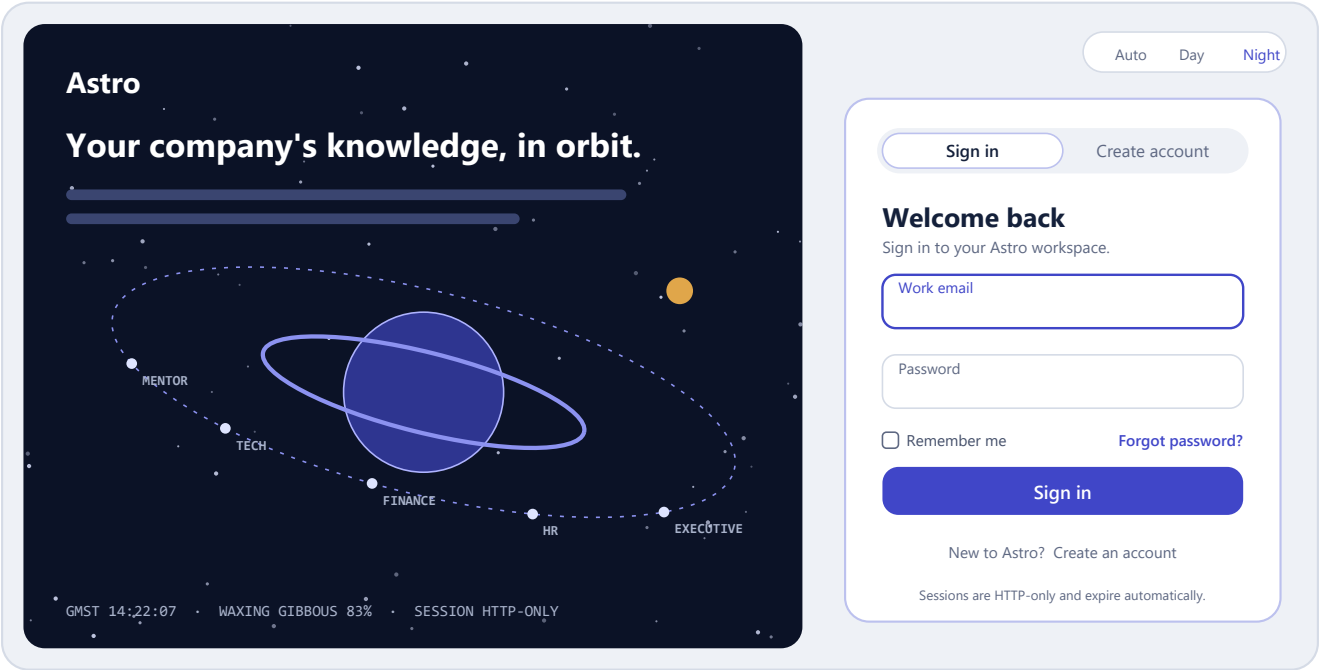


Figure 7. The Login page: the observatory on the left, the auth card on the right. It follows the Day, Night and Auto themes.

The Login page

- **One route, /login, four modes:** Sign in, Create account, Forgot password and Choose a new password (from the email link). Verification links open the same page and sign you in.
- **Observatory:** a slow orrery with the five agents on the outer ring, moons on inner orbits, the Astro mark, a headline, and a live telemetry line (sidereal time, moon phase) reused from the sidebar.
- **Auth card:** a sliding Sign in / Create account switch, floating-label fields, show and hide password, a constellation strength meter (five stars light up), Remember me, and a button with an orbit spinner.
- **After sign-up or forgot password:** “Check your inbox” with a transmitting-satellite animation, a resend button with a cooldown, and in development a link to the local mailbox.
- Works at phone width, with keyboard and screen readers (labels, live error messages, focus order), and respects reduced motion. After sign-in you return to the page you were opening.

Files in astro-web

File	Change
<code>src/api/http.ts</code>	New: the api() wrapper (cookies, JSON, error shape, 401 → /login)
<code>src/api/auth.ts</code>	New: signup, verify, login, logout, forgot, reset, me
<code>src/auth/AuthProvider.tsx</code> · <code>RequireAuth.tsx</code>	New: loads /me once; guards the app and the admin pages
<code>src/pages/AuthPage.tsx</code> · <code>.module.scss</code>	New: the Login page
<code>src/components/Orrery/</code> · <code>AuthField/</code> · <code>PasswordStrength/</code>	New: observatory, field, strength meter

File	Change
src/App.tsx · src/config.ts	Changed: /login route and guards; config keeps only the mock session
AppShell and 7 pages that read company or currentUser	Changed: read the real session; sign-out in the sidebar
.env.example · README.md	Changed: per-area mock flags (VITE MOCK CHAT, VITE MOCK MEETINGS, ...) and the auth contract

Screens go live in this order

Screen	Goes live at
Login, sidebar user and company	CP0
Setup (connect sources), Knowledge	CP1 and CP4
Ask, Voice call (browser speech)	CP2
Agents	CP7
Briefs, Voice (server speech)	CP8
Meetings, Live meeting, Recap	CP9 to CP11

PART C · DELIVERY

SECTION 20

Testing, evals and safety

The permission and cross-tenant suites must pass at 100%, not “most” (roadmap \$15, CP4).

Suite	What it proves	When
Unit	Tools, SQL gateway, chunking, parsers, approvals, token and cookie helpers	Every pull request
Auth flows	Sign-up, verify, sign-in, sign-out, reset, sessions and every error path	Every pull request
Cross-tenant	Signs in as company A and calls every route that takes a company or object ID with company B's IDs; must get 403 or 404 and no data. Routes are listed from the app, so new ones are covered automatically	Every pull request
RLS	Raw SQL as astro_app sees only the company in app.company_id, and nothing when it's unset	Every pull request
Permissions	Role × domain and source ACLs hold for every agent, including Executive calling others	Every pull request
Prompt injection	Documents with hidden instructions can't make an agent call tools, leak data or change rules	Every pull request
Security	Cookie flags, headers, Origin check, identical replies and timing for unknown emails	Every pull request
Migrations	Upgrade to head and back down on an empty database	Every pull request
Evals	Retrieval hit rate, correct answers with valid citations, routing accuracy; 50 to 100 real questions per design partner	Nightly and before releases
Meeting replay	Recorded test meetings produce the same decisions and actions	Before meetings releases
Load	The scenarios in \$2.5	Before each checkpoint and design partner

CI pipeline

ruff → mypy → start Postgres 16 (pgvector) and Redis as service containers → create the roles → alembic upgrade → pytest with coverage → build the Docker image → push to GitHub Container Registry on main → deploy to staging (once hosting is chosen) → production on a version tag. Tests use the stub model, so CI never spends on the LLM; nightly evals use the real one.

SECTION 21

Deployment

One image, several commands. Everything is reached through URLs and keys in environment variables, so nothing ties you to one host.

The image

```
# multi-stage: uv installs into a venv in the builder; python:3.12-slim runs it as a non-root user
api      uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers $WEB_CONCURRENCY \
        --proxy-headers --timeout-graceful-shutdown 30
worker   arq app.workers.worker.WorkerSettings      # queues: default, ingestion, agents, email
bot      arq app.workers.bot_worker.BotWorkerSettings # phase 3
migrate  alembic upgrade head                      # one-off job before each rollout
```

Environments

Environment	Setup
Local	docker compose (Postgres + pgvector, PgBouncer, Redis, MinIO, Mailpit, API, worker), or without Docker: a local or Neon Postgres with in-memory fallbacks for Redis. Web on :5173 through the Vite proxy.
Staging	The same image on your host; seeded demo companies (Kestrel Labs, Northwind Health); a test Google Workspace and GitHub org; real email to a test inbox.
Production	2 or more API containers across zones, autoscaled on CPU and concurrent requests; workers autoscaled on queue depth; managed Postgres with daily backups and point-in-time restore; managed Redis; object storage; secrets manager and KMS.

Start free, then grow

Piece	Start free	Grow to
Postgres	Neon free plan (includes a pooled PgBouncer URL)	Neon paid, AWS RDS or Aurora, GCP Cloud SQL
Redis	Upstash free tier	Upstash paid, ElastiCache, Memorystore
API and workers	A container host with a free or low-cost tier (for example Render, Railway or Fly.io)	AWS ECS Fargate, GCP Cloud Run, or Kubernetes with autoscaling
Object storage	Cloudflare R2 free tier	R2, S3 or GCS
Email	Brevo or Resend free tier (needs a verified sending domain)	Paid tier or Amazon SES
LLM	Stub model for development; pay per use for real answers	Same, with caching and spend caps
Images and CI	GitHub Actions and Container Registry	Same
Errors and LLM traces	Sentry and Langfuse free tiers, or self-hosted	Paid tiers or self-hosted

FREE TIERS CHANGE

Free plans and their limits change often. Check current terms before relying on one; moving between them is a change of environment variables, not code.

SECTION 22

Build order and checkpoints

The roadmap's phases, with Phase 0 broken into the steps I will build first. Each step ends with something you can see or test.

Phase 0 · Foundations (first)

Step	Build	Done when
P0.1 Skeleton	pyproject, app factory, settings, logging, errors, middleware, health probes, Dockerfile, compose, CI lint	/healthz and /readyz answer; CI is green
P0.2 Database	Async engines, Phase 0 models, migration 0001 with RLS, db_roles.sql, seed of two companies	RLS test passes as astro_app on Postgres
P0.3 Auth core	Sign-up, verify, sign-in, sign-out, sessions, /me, cookies, audit rows	Sign up, verify and sign in from /docs
P0.4 Recovery and limits	Forgot and reset, email jobs and templates, rate limits, sign out everywhere, session list	A reset works end to end through the dev mailbox; limits trip
P0.5 Frontend	Login page, AuthProvider, api(), route guards, /me replacing config.ts, sign-out	You sign in on the web app and see your name and company
P0.6 SSO	Google and Microsoft OIDC	Sign in with Google works (needs your OAuth apps)
P0.7 Prove it	Cross-tenant, RLS, security and migration suites in CI; first Locust run	CP0 passes

The rest of the roadmap

Phase	Weeks	Build	Ends at
1 · Answers with sources	2–4	Connector framework, Drive or Notion, uploads, ingestion; retrieval; agent runner with Mentor; chat stream; GitHub App, Tech, supervisor; access matrix, groups, audit list	CP1–CP4
2 · Specialists and admin	5–8	Finance and the SQL gateway; HR and Executive; agent configs, approvals, Jira; briefs scheduler; optional server speech	CP5–CP8
3 · Meetings	9–12	Calendar, bot provider, join and introduce; streaming STT, speakers, guests, wake word, answers; notes, recap, approvals, Slack; design-partner pilot	CP9–CP12
4 · Harden and scale	after 12	SAML and SCIM; more connectors and the relay; Zoom and Teams; dedicated deployments; security programme; admin analytics	ongoing

ONE CHANGE FROM THE CODE STARTED EARLIER

The partial astro-api files written before this plan (synchronous, SQLite by default) are replaced by this async Postgres design in step P0.1. Nothing else in astro-web changes until P0.5.

SECTION 23

What you set up and what I do

I write and run everything I can on this machine. A few things need your accounts, and none of them block the start.

Item	Who	When	Notes
All backend and frontend code, Dockerfile, compose, CI, tests, seed data, docs	Me	Every step	
Python 3.12	Done		Installed for your Windows user
Postgres for development	You pick, I set it up	Before P0.2	(a) Neon: create a free project and send me the connection string; I create the roles. (b) I install PostgreSQL 16 with winget and you approve one admin prompt. (c) Install Docker Desktop and I run the whole stack with compose
Redis	Not needed locally	Before staging	Upstash free database; send me the URL
Email	Not needed locally	Before real users	Brevo or Resend account, verify your sending domain, send the SMTP details
Anthropic API key	You	Before CP2	Tests and development use the stub model until then
Google and Microsoft sign-in	You create the OAuth apps; I wire them	P0.6	I'll give you the exact redirect URLs
Connector apps (Drive, Notion, GitHub App, Atlassian, Slack)	You register; I build	Phase 1 onwards	Free to register; step-by-step notes for each
GitHub repository	You	For CI	I add the workflows; you add the secrets
Hosting account and domain	You	Before staging	Pick after Phase 0; the image runs anywhere
Meeting-bot provider, speech provider	You	Phase 3	Compare current pricing then

SECTION 24

Decisions for you

Answer these and I start at P0.1.

NEEDED NOW

- **Development database:** Neon (cloud, free, nothing to install), local PostgreSQL through winget, or Docker Desktop? I recommend Neon: it is the same kind of pooled Postgres you would run in production.
- **Email verification before the first sign-in:** keep it (recommended; it stops people joining another company's workspace with a fake address) or allow sign-in first?
- **Approve this plan,** or tell me what to change.

CAN WAIT

- Email provider and sending domain: before real users.
- Hosting: before staging.
- LLM key: before CP2.
- Meeting-bot and speech providers: before Phase 3.

APPENDIX A

All API endpoints

Everything lives under /api and is scoped to the signed-in user's company. “UI” means the frontend already calls it.

Endpoint	Purpose	Phase
POST /auth/signup • /verify-email • /resend-verification	Accounts	0
POST /auth/login • /logout • /logout-all	Sessions	0
POST /auth/forgot-password • /reset-password	Password recovery	0
GET /auth/sessions • DELETE /auth/sessions/{id}	Your devices	0
GET /auth/{provider}/start • /callback	Google and Microsoft sign-in	0
GET /me	Current user, company, roles	0
GET /companies/{id} • /companies/{id}/members	Company and members	0
GET /audit	Access checks and admin events	0 • 1d
POST /chat/stream	Ask, SSE route/agent/token/sources/done/error (UI)	1b
GET /conversations • /conversations/{id}	Chat history	1b
POST /feedback	Thumbs up or down	1b
GET /companies/{id}/sources (UI) • DELETE /sources/{id}	Sources and status	1a
GET /connectors/{kind}/start • /callback • POST /uploads	Connect sources, upload files	1a
PUT /companies/{id}/access (UI)	Role × domain matrix	1d
GET /companies/{id}/agents (UI) • PATCH .../agents/{agentId} (UI)	Agent configs, pause, guardrails	2c
POST /agents/route-test	Try the supervisor	2c
GET /approvals • POST /approvals/{id}/approve • /reject	Pending actions	2c
GET /briefs • /briefs/{id} • POST .../regenerate • PUT /brief-schedules	Briefs	2d
GET /companies/{id}/meetings (UI) • POST .../meetings/join (UI)	Meetings, send Astro by link	3a
PATCH /companies/{id}/meetings/{meetingId} (UI) • PUT .../meeting-settings (UI)	Invite switch, settings	3a
GET /meetings/{id}/live (UI) • PATCH .../live (UI)	Live SSE stream and controls	3b
POST /meetings/{id}/ask-private (UI) • /leave (UI)	Private question, leave	3b
GET /meetings/{id}/recap (UI)	Recap	3c
POST /meetings/{id}/actions/{actionId}/approve (UI) • /follow-up (UI)	Approvals, Slack follow-up	3c
WS /voice	Streaming speech	2d
GET /healthz • /readyz • /metrics	Probes and metrics (not public)	0

APPENDIX B

Settings

Every setting has a safe local default in `.env.example`. Production values come from the host's secret manager.

Variable	Purpose
APP_ENV	development, test or production; production turns on the strict checks
DATABASE_URL · AUTH_DATABASE_URL	astro_app and astro_auth through PgBouncer
MIGRATIONS_DATABASE_URL	Owner role, direct connection, for Alembic only
DB_POOL_SIZE · DB_MAX_OVERFLOW · DB_STATEMENT_TIMEOUT_MS	Pool and query limits per process
REDIS_URL	Cache, limits, queues, pub/sub; required in production
WEB_ORIGIN · EXTRA_ORIGINS	Where emailed links point; allowed origins for writes
SESSION_TTL_HOURS · SESSION_REMEMBER_DAYS	12 and 30 by default
VERIFY_TOKEN_HOURS · RESET_TOKEN_MINUTES	48 and 30 by default
SMTP_HOST · SMTP_PORT · SMTP_USER · SMTP_PASSWORD · EMAIL_FROM	Email; empty host means console and dev mailbox
WEB_CONCURRENCY	Uvicorn processes per container
ANTHROPIC_API_KEY · LLM_ANSWER_MODEL · LLM_ROUTER_MODEL · LLM_STUB	claude-opus-5 and claude-haiku-4-5 by default; the stub for tests
EMBEDDINGS_URL · RERANK_URL	Embedding and reranking services
S3_ENDPOINT · S3_BUCKET · S3_ACCESS_KEY · S3_SECRET_KEY	Object storage
KMS_KEY_ID · LOCAL_MASTER_KEY	Envelope encryption; the local key is for development only
GOOGLE_CLIENT_ID/SECRET · MICROSOFT_CLIENT_ID/SECRET/TENANT	SSO (P0.6)
SENTRY_DSN · OTEL_EXPORTER_OTLP_ENDPOINT · LANGFUSE_*	Errors, traces, agent-run tracing